Zephyr RTOS como plataforma agnóstica al hardware

Devicetree y drivers estandarizados para portar aplicaciones embebidas sin modificar su código

Zephyr RTOS as a hardware-agnostic platform

Devicetree and standardized drivers to port embedded applications without changing their source code

[Fabrizio Carlassara][1] *

[1] Universidad Tecnológica Nacional, Facultad Regional Avellaneda, Laboratorio de Sistemas Embebidos, Avellaneda, Buenos Aires, Argentina · ORCID:

Autor de correspondencia: fabrizio.carlassara@gmail.com

Resumen

El firmware para microcontroladores suele quedar atado al kit de desarrollo de cada fabricante, lo que encarece la migración entre plataformas ante faltantes de componentes o cambios de requerimientos. Esta nota técnica analiza los mecanismos con los que Zephyr, un sistema operativo de tiempo real de código abierto alojado por la Linux Foundation, desacopla la aplicación del hardware: el devicetree resuelto en tiempo de compilación, el modelo de dispositivos con interfaces de programación estandarizadas y el sistema de configuración Kconfig. Para evaluarlos se desarrollaron dos aplicaciones, una de entradas y salidas digitales con interrupciones y otra de lectura de acelerómetros por I2C, que se compilaron sin modificar su código fuente para diez placas de cinco fabricantes de silicio y tres arquitecturas de procesador. Las diferencias entre plataformas quedaron confinadas a archivos de superposición de devicetree de pocas líneas, y el reemplazo de un acelerómetro integrado en la placa por uno externo se resolvió con el mismo mecanismo. Se informa el uso de memoria de cada plataforma y se discuten las limitaciones del enfoque. Se concluye que Zephyr permite tratar la placa como un parámetro de compilación, con beneficios para el desarrollo de productos y para la enseñanza.

Palabras clave: sistemas embebidos; sistema operativo de tiempo real; Zephyr; devicetree; portabilidad

Abstract

Microcontroller firmware is usually tied to each vendor's software development kit, which makes migrating between platforms costly when components become scarce or requirements change. This technical note analyzes the mechanisms through which Zephyr, an open-source real-time operating system hosted by the Linux Foundation, decouples the application from the hardware: a devicetree resolved at build time, a device model with standardized programming interfaces, and the Kconfig configuration system. To evaluate them, two example applications

were developed, one for digital input and output with interrupts and one for reading accelerometers over I2C, and they were built without modifying their source code for ten boards from five silicon vendors and three processor architectures. Platform differences were confined to short devicetree overlay files, and replacing an on-board accelerometer with an external one was handled with the same mechanism. Memory usage on each platform is reported, and the limitations of the approach are discussed. It is concluded that Zephyr makes it possible to treat the board as a build parameter, with benefits for product development and for engineering education.

Keywords: embedded systems; real-time operating system; Zephyr; devicetree; portability

1. Introducción

Un proyecto de firmware para microcontroladores se construye habitualmente sobre el kit de desarrollo de software (SDK) que provee el fabricante del circuito integrado: su capa de abstracción de hardware (HAL), su sistema de compilación y, con frecuencia, su entorno de desarrollo. Esta elección, razonable al comienzo de un proyecto, mezcla en los mismos archivos la lógica de la aplicación con detalles de la plataforma, como números de pin, nombres de periféricos y secuencias de inicialización de relojes. Cuando el microcontrolador debe reemplazarse, por faltante de stock, por costo o porque cambian los requerimientos, el resultado práctico es reescribir buena parte del firmware.

Los sistemas operativos de tiempo real (RTOS) tradicionales para microcontroladores, como FreeRTOS [1], resuelven la planificación de tareas y la sincronización, pero dejan el acceso a periféricos en manos del SDK de cada fabricante. Las comparaciones entre RTOS suelen centrarse en el desempeño temporal del núcleo [2], mientras que la portabilidad de la aplicación entre plataformas recibe menos atención.

Zephyr es un RTOS de código abierto, distribuido bajo licencia Apache 2.0 y alojado por la Linux Foundation, que adopta otro enfoque: además del núcleo, incluye un modelo de drivers propio con interfaces estandarizadas, una descripción declarativa del hardware basada en devicetree y un sistema de compilación y configuración unificado [3]. En la versión 4.4.2, utilizada en este trabajo, el árbol de código fuente contiene definiciones para 1014 placas de 160 fabricantes y once arquitecturas de procesador.

El objetivo de esta nota técnica es describir esos mecanismos y evaluar, con aplicaciones concretas, en qué medida permiten que una misma aplicación se compile y ejecute en plataformas de distintos fabricantes sin modificar su código fuente. El trabajo se enmarca en las actividades del Laboratorio de Sistemas Embebidos de la UTN FRA y acompaña la charla homónima presentada en el Congreso de Ingeniería Electrónica 2026.

2. Desarrollo

2.1. Arquitectura de Zephyr

Zephyr organiza el software en capas. La aplicación utiliza los servicios del núcleo (hilos, planificador, semáforos, colas, temporizadores) y las *interfaces de programación (API) de dispositivos*, que son genéricas para cada clase de periférico: GPIO, I^2C, SPI, UART, ADC, PWM y sensores, entre otras. Debajo de cada API hay uno o varios drivers, cada uno específico de un periférico o familia de circuitos integrados; algunos se apoyan internamente en la HAL del fabricante, que queda oculta para la aplicación.

Dos sistemas de configuración, ambos resueltos durante la compilación, determinan qué se incluye en la imagen final:

- el **devicetree**, que describe *qué hardware existe y cómo está conectado*;
- **Kconfig**, que define *qué componentes de software se compilan* y con qué parámetros [4].

2.2. Devicetree

El devicetree es una estructura de datos jerárquica para describir hardware, especificada por la comunidad devicetree.org [5] y adoptada por el núcleo Linux para eliminar del código fuente las descripciones específicas de cada placa [6]. Cada *nodo* representa un componente (un controlador I²C, un sensor, un LED) y contiene *propiedades*. La propiedad `compatible` identifica qué es el dispositivo y, por lo tanto, qué driver lo atiende; `reg` indica su dirección y `status` si está habilitado.

La descripción se arma por capas. El fabricante del SoC provee un archivo `.dtsi` con todos sus periféricos; quien diseña la placa escribe un `.dts` que lo incluye y declara lo que la placa agrega (LED, pulsadores, cristal, pines utilizados); y la aplicación puede aportar archivos de superposición (*overlays*) que agregan o modifican nodos sin tocar los anteriores. El Listado 1 muestra un overlay que conecta un acelerómetro MPU6050 a un bus I²C y le asigna el alias genérico `sensor0`.

Listado 1: Overlay que agrega un MPU6050 al bus I²C definido por la placa.

```
&sensor_i2c {
    mpu6050: mpu6050@68 {
        compatible = "invensense,mpu6050";
        reg = <0x68>;
    };
};
/ { aliases { sensor0 = &mpu6050; }; };
```

A diferencia de Linux, donde el árbol se interpreta durante el arranque, Zephyr lo procesa durante la compilación [7]. Un conjunto de scripts en Python combina las capas, valida cada nodo contra su *binding* (un archivo YAML que define las propiedades admitidas para cada `compatible`) y genera el encabezado `devicetree_generated.h`, con macros de C para cada nodo y propiedad. La aplicación accede a esa información mediante macros como `DT_ALIAS()`, `DT_NODELABEL()` o `GPIO_DT_SPEC_GET()`, que se resuelven en constantes. En consecuencia, la descripción del hardware no ocupa memoria de datos ni tiempo de ejecución, y los errores, como un pin inexistente o una propiedad obligatoria ausente, se detectan al compilar.

Los *alias* (`led0`, `sw0`, `sensor0`) constituyen la interfaz entre la aplicación y la placa: la aplicación los referencia por nombre y cada placa, o su overlay, decide a qué nodo físico apuntan.

2.3. Modelo de dispositivos y API estandarizadas

Cada nodo del devicetree habilitado y con driver disponible da lugar a una instancia de `struct device`, creada en tiempo de compilación [8]. La estructura contiene un puntero a la configuración constante (derivada del devicetree y ubicada en memoria no volátil), uno a los datos de estado (en RAM) y uno a una tabla de funciones, la *API* del driver. Los dispositivos se inicializan automáticamente antes de `main()`, por niveles y prioridades que respetan las dependencias (por ejemplo, el bus I²C antes que el sensor conectado a él).

Las funciones públicas de cada clase de dispositivo, como `gpio_pin_toggle_dt()` o `sensor_sample_fetch()`, son funciones *inline* que invocan la entrada correspondiente de esa tabla. La implementación que se ejecuta queda determinada por el devicetree, sin compilación condicional en el código de la aplicación. En el caso de los sensores [9], la API define *canales* con unidades del Sistema Internacional (aceleración en $\mathrm{m\,s^{-2}}$, velocidad angular en $\mathrm{rad\,s^{-1}}$, temperatura en $^{\circ}\mathrm{C}$) y valores de punto fijo (`struct sensor_value`); un driver que no implementa un canal devuelve `-ENOTSUP`.

Kconfig complementa el modelo: la aplicación declara en `prj.conf` las API que usa (`CONFIG_GPIO`, `CONFIG_I2C`, `CONFIG_SENSOR`) y los drivers concretos se habilitan solos cuando el devicetree contiene un nodo habilitado con el `compatible` correspondiente. Un archivo opcional `boards/<placa>.conf` se combina con `prj.conf` solo al compilar para esa placa, lo que permite ajustar el software a una plataforma sin tocar la configuración común.

2.4. Diseño experimental

Se construyó un espacio de trabajo con topología de *manifest* propio de la herramienta `west` [10], que fija Zephyr en la versión 4.4.2 y descarga solo los módulos de HAL necesarios. Sobre él se desarrollaron dos aplicaciones:

- **blinky_boton**: un LED parpadea y cada flanco del pulsador, atendido por interrupción, cambia el período. Utiliza solo los alias `led0` y `sw0` y la API de GPIO.
- **acelerometro**: obtiene el dispositivo apuntado por el alias `sensor0` y, cada dos segundos, solicita aceleración, velocidad angular, campo magnético y temperatura del chip, mostrando las magnitudes que el driver responda. No contiene ninguna referencia a un sensor o placa en particular. Incluye además la consola interactiva (*shell*) de Zephyr, con comandos para inspeccionar dispositivos, buses $\mathrm{I^2C}$ y sensores.

Para la aplicación del acelerómetro los overlays se separaron en dos ejes independientes. El overlay de cada placa, que el sistema de compilación aplica automáticamente, define qué bus $\mathrm{I^2C}$ se usa, asignándole la etiqueta común `sensor_i2c`, y, si la placa integra un acelerómetro, lo designa como `sensor0`. Dos de las placas evaluadas lo integran: el FXOS8700 de la FRDM-K64F (acelerómetro y magnetómetro) y el MMA8652FC de la LPCXpresso55S69, ambos atendidos por el mismo driver. El overlay del MPU6050 (Listado 1) se solicita en la invocación de compilación (`-DEXTRA_DTC_OVERLAY_FILE=mpu6050.overlay`); como se aplica después del de la placa, reemplaza al sensor integrado cuando lo hay. De este modo, N placas y M sensores requieren $N + M$ archivos en lugar de $N \times M$.

La Tabla 1 resume las plataformas evaluadas. Cada combinación se compiló desde la consola con `west build`, a partir de un directorio limpio, y se registraron el resultado y la memoria estática informada por el enlazador. Se utilizó el Zephyr SDK 1.0.1 [11] en una computadora portátil con procesador Apple M3 Pro y macOS.

Tabla 1: Plataformas evaluadas

Placa	Fabricante del SoC	SoC	Núcleo	Acelerómetro integrado
Raspberry Pi Pico	Raspberry Pi	RP2040	Cortex-M0+	–
Raspberry Pi Pico 2	Raspberry Pi	RP2350A	Cortex-M33	–
WeAct Black Pill F401	STMicroelectronics	STM32F401CC	Cortex-M4F	–
DOIT ESP32 DevKit V1	Espressif	ESP32	Xtensa LX6	–
ESP32-C3 SuperMini	Espressif	ESP32-C3	RISC-V (RV32IMC)	–
ESP32-S3 DevKitC	Espressif	ESP32-S3	Xtensa LX7	–
nRF5340 DK	Nordic Semiconductor	nRF5340	Cortex-M33	–
LPCXpresso55S69	NXP	LPC55S69	Cortex-M33	MMA8652FC
FRDM-MCXN236	NXP	MCXN236	Cortex-M33	–
FRDM-K64F	NXP	MK64FN1M0	Cortex-M4F	FXOS8700

3. Resultados

Se realizaron 22 compilaciones, todas exitosas: `blinky_boton` para las diez placas, `acelerometro` con el MPU6050 para las diez y `acelerometro` con el sensor integrado para las dos placas que lo tienen. En ninguna se modificó el archivo `main.c` ni el `prj.conf`. La Tabla 2 presenta, para la aplicación del acelerómetro con MPU6050, el uso de memoria y la cantidad de líneas efectivas de overlay propias de cada placa, frente a las 61 líneas efectivas de la aplicación.

Tabla 2: Aplicación *acelerometro* con MPU6050: memoria y líneas de overlay por placa

Placa	Núcleo	Flash (KiB)	RAM (KiB)	Overlay placa	Overlay total
Raspberry Pi Pico	Cortex-M0+	90,7	17,0	3	14
Raspberry Pi Pico 2	Cortex-M33	86,1	17,2	3	14
WeAct Black Pill F401	Cortex-M4F	91,3	18,0	3	14
DOIT ESP32 DevKit V1	Xtensa LX6	185,1	62,9	3	14
ESP32-C3 SuperMini	RISC-V	148,0	112,3	3	14
ESP32-S3 DevKitC	Xtensa LX7	148,5	110,8	3	14
nRF5340 DK	Cortex-M33	92,4	17,6	3	14
LPCXpresso55S69	Cortex-M33	90,9	17,4	8	19
FRDM-MCXN236	Cortex-M33	93,1	17,7	3	14
FRDM-K64F	Cortex-M4F	92,3	17,9	8	19

Líneas efectivas (sin comentarios ni líneas vacías). `main.c`: 61 líneas, idéntico en todas las placas. Overlay total = overlay de la placa + `mpu6050.overlay` (11 líneas). Flash y RAM: memoria estática informada por el enlazador; en ESP32 la RAM suma IRAM y DRAM. Incluye núcleo, drivers, consola y *shell*.

Para `blinky_boton`, 7 de las 10 placas no requirieron overlay, porque su devicetree ya define `led0` y `sw0`. Las restantes solo necesitaron declarar un LED o un pulsador externo, en no

Tabla 3: Magnitudes informadas por la misma aplicación según el sensor

Sensor	Ubicación	Aceleración	Giro	Campo magn.	Temp. chip
MPU6050	externo, cualquier placa	✓	✓	–	✓
FXOS8700	integrado, FRDM-K64F	✓	–	✓	–
MMA8652FC	integrado, LPCXpresso55S69	✓	–	–	–

más de 14 líneas.

La Tabla 3 muestra cómo la misma aplicación se adapta al sensor disponible. En la FRDM-K64F, pasar del FXOS8700 integrado al MPU6050 externo solo requirió agregar `mpu6050.overlay` a la invocación de compilación: la aplicación dejó de informar campo magnético y pasó a informar velocidad angular y temperatura del chip. Los dos acelerómetros integrados, de distinto modelo, son atendidos por el mismo driver; el modo de operación (solo acelerómetro, o acelerómetro y magnetómetro) lo fija la configuración que Zephyr define para cada placa.

En las placas Cortex-M, la aplicación ocupó entre 86,1 y 93,1 KiB de memoria de programa y entre 17 y 18 KiB de RAM, valores dominados por el núcleo, los drivers, el registro de eventos y la *shell*, no por la aplicación. Las imágenes para los SoC de Espressif, tanto Xtensa como RISC-V, resultaron mayores por el código de arranque y de soporte propio de esos chips.

4. Discusión

Los resultados muestran que, para las clases de periféricos evaluadas, el devicetree y las API estandarizadas trasladan las diferencias entre plataformas desde el código de la aplicación a archivos declarativos breves, validados en compilación. El costo de portar una aplicación a una placa nueva pasa a ser el de describir sus conexiones, y la separación en overlays de placa y de sensor evita la explosión combinatoria de configuraciones. La consulta de canales con respuesta `-ENOTSUP` permite, además, que una sola aplicación se adapte a sensores con capacidades distintas sin conocerlos.

El enfoque tiene limitaciones que conviene explicitar. Primero, la portabilidad alcanza a lo que cubren las API comunes: periféricos exclusivos de un fabricante, como el PIO del RP2040 o la radio de los nRF, se exponen con interfaces propias o con un subconjunto de funcionalidad. Segundo, no todos los drivers implementan todas las capacidades opcionales de su API, como transferencias por DMA, operación asíncrona o modos de bajo consumo. Tercero, el uso de memoria es mayor que el de una solución sin sistema operativo o con un núcleo mínimo: con el registro de eventos (*logging*) y la *shell*, la aplicación necesita unos 90 KiB de flash y 18 KiB de RAM en las placas Cortex-M, lo que deja afuera a los microcontroladores más pequeños o exige recortar funcionalidad mediante Kconfig. Cuarto, compilar en una plataforma no equivale a validarla: la temporización, el consumo y el comportamiento eléctrico siguen dependiendo del silicio y requieren ensayos en el hardware. Por último, la curva de aprendizaje inicial es mayor, ya que se deben incorporar devicetree, Kconfig, CMake y `west`.

En el ámbito educativo, el mismo mecanismo permite que los ejercicios de una asignatura no

dependan de una placa en particular: cada estudiante puede trabajar con el hardware disponible y concentrarse en conceptos (interrupciones, buses, abstracción de dispositivos) en lugar de en las particularidades de un SDK. Como todo el flujo se resuelve con una única herramienta de consola, igual en Linux, macOS y Windows, el mismo laboratorio puede atender placas de distintos fabricantes sin instalar un entorno por cada uno.

5. Conclusiones

Se analizaron los mecanismos con los que Zephyr desacopla la aplicación del hardware y se verificó experimentalmente que dos aplicaciones, de entradas y salidas digitales y de lectura de acelerómetros, se compilan sin cambios en su código para diez placas de cinco fabricantes y tres arquitecturas de procesador (ARM Cortex-M, Xtensa y RISC-V). Las diferencias quedaron confinadas a overlays de devicetree de pocas líneas, y el reemplazo de un acelerómetro integrado por uno externo se resolvió con el mismo mecanismo. Zephyr permite así tratar la placa, y el sensor, como parámetros de compilación, con beneficios concretos ante cambios de plataforma y en la enseñanza de sistemas embebidos.

Como líneas futuras se propone extender la evaluación a otras clases de dispositivos (ADC, PWM, comunicaciones inalámbricas), medir en el hardware el desempeño temporal y el consumo de una misma aplicación en distintas plataformas e incorporar pruebas automáticas sobre hardware real con la herramienta `twister`.

6. Declaraciones Editoriales

Agradecimientos

Al Departamento de Ingeniería Electrónica de la UTN FRA por el equipamiento del Laboratorio de Sistemas Embebidos.

Disponibilidad de datos, materiales o software

El espacio de trabajo completo (manifest de `west`, aplicaciones, overlays y la guía para reproducir las compilaciones) está disponible en https://github.com/utn-fra-lse/cie2026-zephyr bajo licencia Apache 2.0.

7. Referencias

[1] Richard Barry. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.

[2] Ioan Ungurean. Timing comparison of the real-time operating systems for small microcontrollers. *Symmetry*, 12(4):592, 2020.

[3] Zephyr Project. Zephyr project documentation, versión 4.4. [en línea]. Disponible en: https://docs.zephyrproject.org/4.4.0/, 2026. [consulta: octubre de 2026].

[4] Zephyr Project. Configuration system (kconfig). [en línea]. Disponible en: https://docs.zephyrproject.org/latest/build/kconfig/index.html, 2026. [consulta: octubre de 2026].

[5] devicetree.org. *Devicetree Specification, Release v0.4*, 2023. Disponible en: https://www.devicetree.org/specifications/.

[6] The Linux Kernel Documentation. Linux and the devicetree. [en línea]. Disponible en: https://docs.kernel.org/devicetree/usage-model.html. [consulta: octubre de 2026].

[7] Zephyr Project. Devicetree guide. [en línea]. Disponible en: https://docs.zephyrproject.org/latest/build/dts/index.html, 2026. [consulta: octubre de 2026].

[8] Zephyr Project. Device driver model. [en línea]. Disponible en: https://docs.zephyrproject.org/latest/kernel/drivers/index.html, 2026. [consulta: octubre de 2026].

[9] Zephyr Project. Sensors. [en línea]. Disponible en: https://docs.zephyrproject.org/latest/hardware/peripherals/sensor/index.html, 2026. [consulta: octubre de 2026].

[10] Zephyr Project. West (zephyr's meta-tool). [en línea]. Disponible en: https://docs.zephyrproject.org/latest/develop/west/index.html, 2026. [consulta: octubre de 2026].

[11] Zephyr Project. Zephyr sdk. [en línea]. Disponible en: https://github.com/zephyrproject-rtos/sdk-ng. [consulta: octubre de 2026].